

CSE221: Assignment 08 Study Notes

Data Structure: Disjoint Set / Union-Find

Spring 2026 • Based on Shafaet's Blog (shafaetsplanet.com)

1. What is Disjoint Set?

Data structures are one of the most fascinating parts of Computer Science. We can store data in countless ways. Today we explore a brilliant data structure called the Disjoint Set (also known as Union-Find).

Why is it called 'Disjoint Set'?

A disjoint set maintains a collection of non-overlapping (disjoint) subsets of a universe. Each subset has a unique representative. We can efficiently ask: 'Are two elements in the same subset?' and 'Merge two subsets into one.'

Disjoint Set is critically important for implementing algorithms such as:

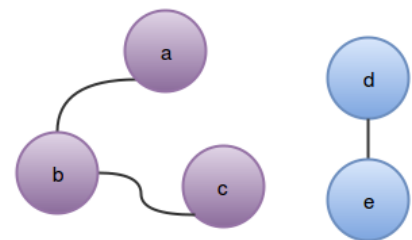
- Kruskal's Minimum Spanning Tree (MST)
- Cycle detection in undirected graphs
- Network connectivity problems

2. Motivating Example: The Friendship Problem

Imagine 5 people: A, B, C, D, E. Friendships are transitive: if A–B are friends and B–C are friends, then A–C are also friends (they belong to the same friend circle).

Suppose we are told:

- A and B are friends
- B and C are friends
- D and E are friends



Now answer these queries:

Query	Answer	Reason
Are A and C friends?	YES ✓	A→B→C are all connected
Are B and D friends?	NO ✗	Different components
Are D and E friends?	YES ✓	Directly connected

We could solve this with BFS/DFS, but Disjoint Set is far more efficient for dynamic updates (adding friendships on the fly).

3. Core Idea: The 4 Principles

The 4 Fundamental Rules of Disjoint Set

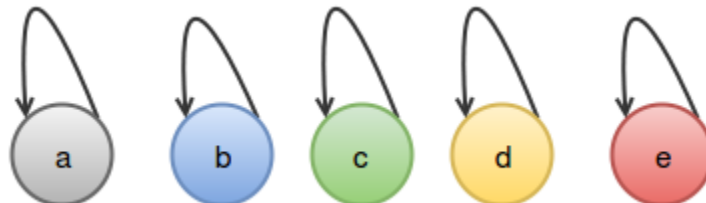
1. People who are friends all belong to the same set
2. Each connected subgraph corresponds to exactly one set.
3. Every set has exactly one representative (the 'root').
4. Two nodes are friends $\Leftrightarrow$ they share the same representative.

Key insight: we use a single array called `par[]` (parent). Each element stores its parent node. The root of a tree is its own parent: `par[root] = root`. This forms a tree/forest structure.

3.1 Initial State: Everyone is their own representative

Initially, no friendships exist. Each person is in their own set and is their own representative:

Index	1	2	3	4	5
<code>par[]</code>	1	2	3	4	5



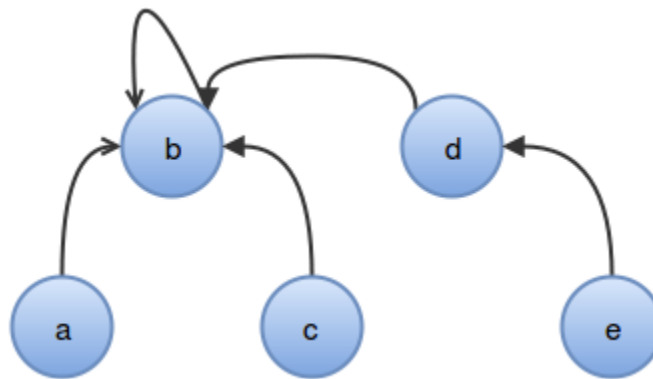
```
int[] par = new int[N + 1];
int[] size = new int[N + 1]; // for tracking component size

for (int i = 1; i <= N; i++) {
    par[i] = i;           // everyone is their own parent
    size[i] = 1;         // every set has size 1
}
```

3.2 Union Operation: Making Two People Friends

When A and B become friends, we merge their sets. A adopts B's representative as its own representative. Node A joins B's tree by setting $\text{par}[\text{find}(A)] = \text{find}(B)$.

Operation	Tree Structure	par[] array
Initial	(1) (2) (3) (4) (5)	[1, 2, 3, 4, 5]
union(1,2) a-b friends	(2) ← (1) (3) (4) (5)	par[1]=2, [2, 2, 3, 4, 5]
union(1,3) a-c friends	(2) ← (1), (3) (4) (5)	par[3]=2, [2, 2, 2, 4, 5]
union(4,5) d-e friends	(2) ← (1), (3) (4) ← (5)	par[5]=4, [2, 2, 2, 4, 4]
union(3,5) e-c friends	(2) ← (1), (3), (4) ← (5)	par[4]=2, [2, 2, 2, 2, 4]



4. The Three Operations

4.1 makeSet(n): Create a new set

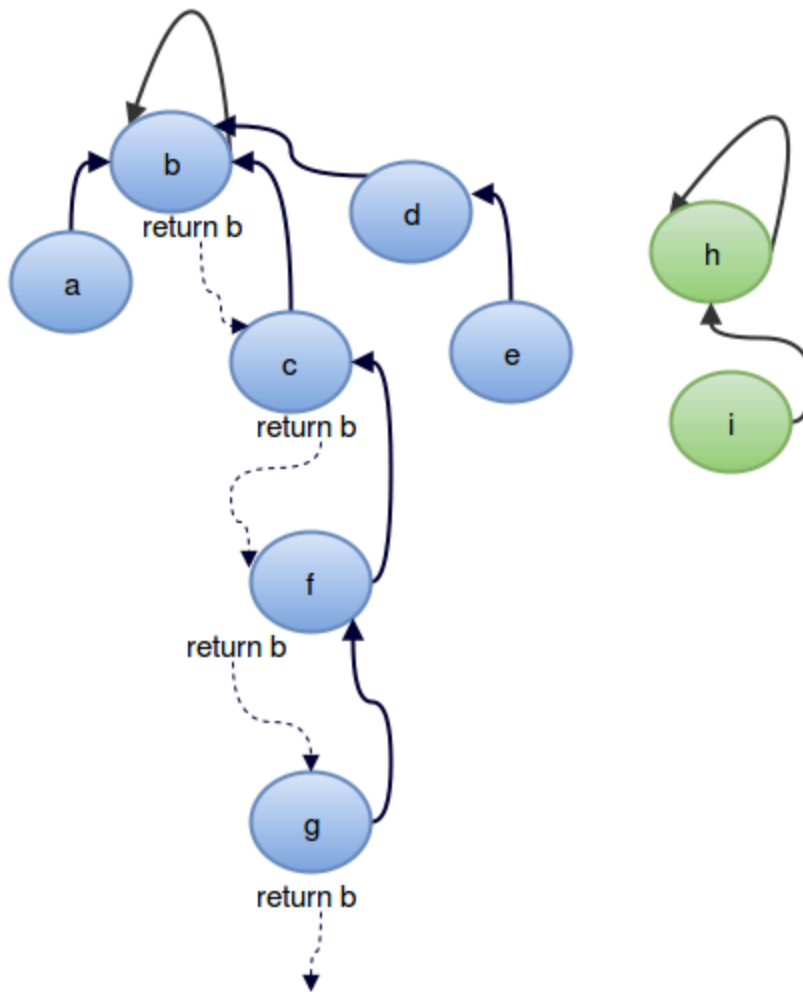
```
void makeSet(int n) {  
    par[n] = n;    // node is its own parent (representative)  
    size[n] = 1;   // set of size 1  
}
```

4.2 find(r): Find the representative of a set

Recursively walk up the parent chain until we reach a node that is its own parent ($\text{par}[r] == r$). That node is the representative/root.

Example: $\text{find}(g)$ in a chain $g \rightarrow f \rightarrow c \rightarrow b$ where $\text{par}[b]=b$:

find(g)	find(f)	find(c)	returns b
---------	---------	---------	------------------



```
// Basic find - O(N) worst case
int find(int r) {
    if (par[r] == r) return r; // base case: found root
    return find(par[r]);      // recurse up
}
```

4.3 union(a, b): Merge two sets

Find the representatives of both nodes. If they are different, attach one root to the other. If they are already the same, they are already in the same set.

```

void union(int a, int b) {
    int u = find(a);
    int v = find(b);
    if (u == v) {
        System.out.println('Already in same set!');
        return;
    }
    par[u] = v;    // OR par[v] = u - both valid
}

```

5. Optimizations

Why optimize?

The basic find() runs in $O(N)$ worst case if the tree becomes a long chain (like a linked list).

5.1 Path Compression

The key insight: when we call find(g) and it returns root b, all intermediate nodes (f, c, etc.) could be directly connected to b. They all share the same representative anyway!

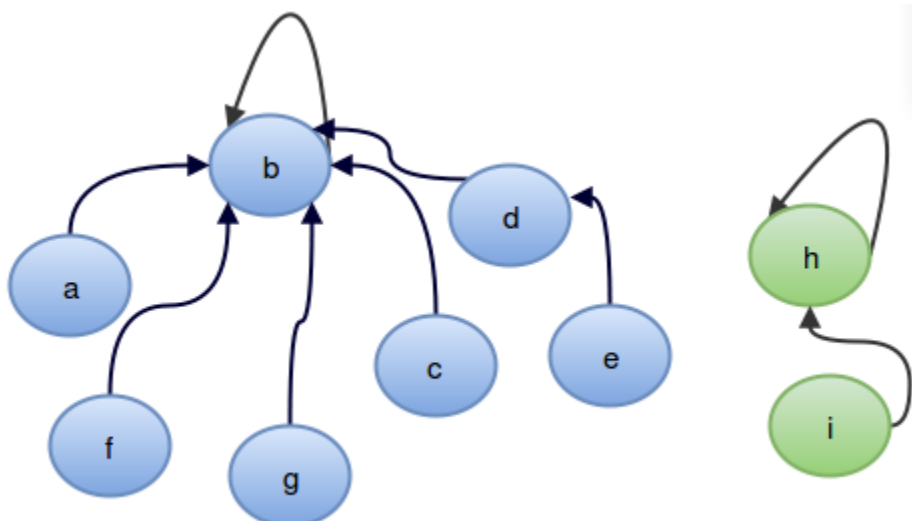
On each find() call, we set every visited node's parent directly to the root. The tree becomes flat:

BEFORE find(g)

$b \leftarrow c \leftarrow f \leftarrow g$
depth = 4 (slow!)

AFTER find(g)

$b \leftarrow c, f, g$ (all direct)
depth = 1 (fast!)



```
// find() WITH path compression
int find(int r) {
    if (par[r] == r) return r;
    par[r] = find(par[r]);    // KEY: set parent directly to root
    return par[r];
}
```

Path compression is a way of flattening the structure of the tree whenever Find is used on it. Each node visited on the way to the root may as well be attached directly to the root. They all share the same representative.

5.2 Union by Size (Rank)

When merging two sets, always attach the smaller tree under the larger tree. This prevents the tree from growing tall.

```
void union(int a, int b) {
    int u = find(a);
    int v = find(b);
    if (u == v) return;
    // Merge smaller set into larger set
    if (size[u] < size[v]) {
        int temp = u; u = v; v = temp; // swap so u is larger
    }
    par[v] = u;           // attach smaller (v) under larger (u)
    size[u] += size[v];   // update size
}
```

6. Complete Pseudocode Summary

DISJOINT SET – Complete Pseudocode

INITIALIZE(N):

```
for i = 1 to N:
    par[i] = i
    size[i] = 1
```

FIND(r): // returns representative of r's set

```
if par[r] == r:
    return r
par[r] = FIND(par[r]) // path compression
return par[r]
```

UNION(a, b): // merges sets containing a and b

```
u = FIND(a)
v = FIND(b)
if u == v:
    return // already same set
if size[u] < size[v]:
    swap(u, v) // u = larger set
par[v] = u // attach smaller under larger
size[u] += size[v]
```

IS_SAME_SET(a, b):

```
return FIND(a) == FIND(b)
```